COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 19
QUERY PROCESSING: COST ESTIMATES

QUERY PROCESSING: OUTLINE

Query Processing Overview

Cost Estimates

Size Estimates

Query Optimization

QUERY PROCESSING OVERVIEW

Parsing and Translation

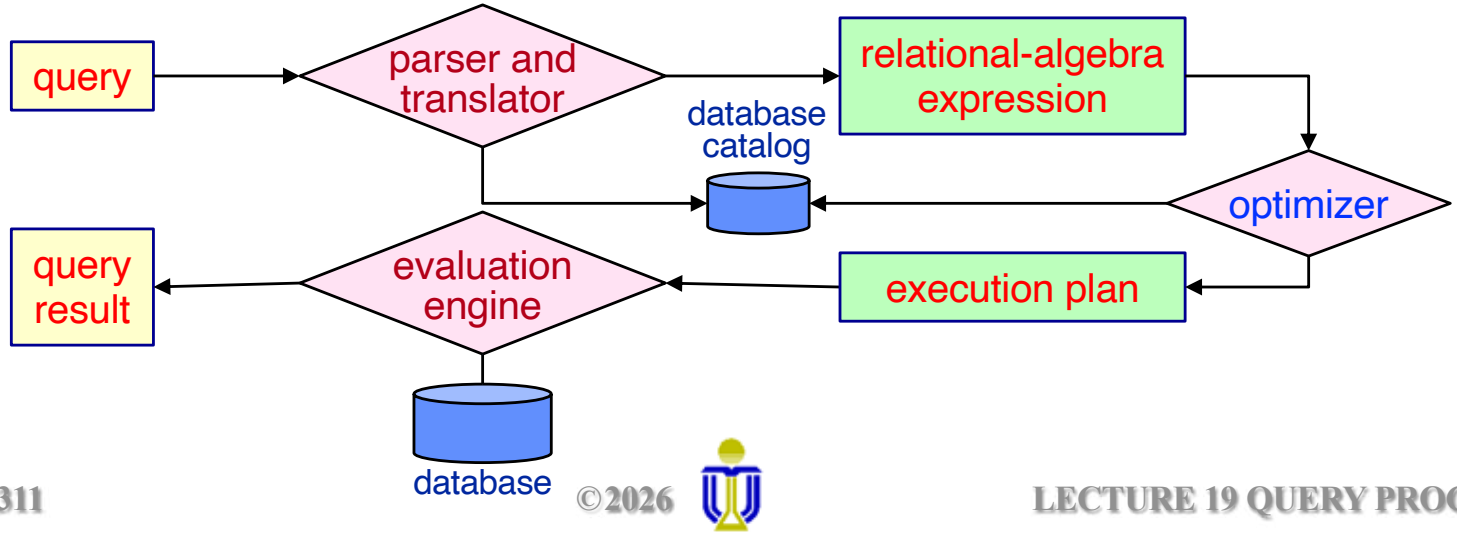
Using the database catalog, the parser and translator first checks the query for syntactic and semantic errors and then translates it into a relational-algebra expression.

Optimization

The optimizer takes the relational-algebra expression and transforms it into an optimal execution plan.

Evaluation

The evaluation engine takes an execution plan, executes that plan by accessing the database, and returns the query result.



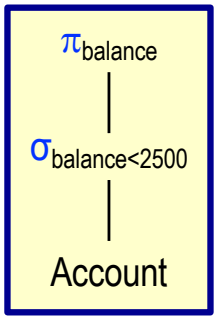
QUERY EVALUATION

- A given relational-algebra expression may have many equivalent expressions.

$$\sigma_{balance<2500}(\pi_{balance}(Account)) \equiv \pi_{balance}(\sigma_{balance<2500}(Account))$$

☞ **An operation in a relational-algebra expression can be evaluated using one of several different algorithms usually having different cost.**

- A query-execution plan represents a relational-algebra expression as a relational-algebra tree and annotates it with a detailed evaluation strategy *for each operation*.



relational-algebra tree

- To process the above query, the query-execution plan can:
  - use an index on balance to find accounts with balance<2500.
    or
  - perform a complete file scan and discard accounts with balance≥2500.

Which query-execution plan to use may depend on several factors.

The query processor chooses, amongst all equivalent evaluation plans, the one with the (expected) lowest cost.

QUERY PROCESSING COST MEASURES

The cost to process a query depends on the cost of each operation in the query-execution plan.

- Cost is estimated using information from the database catalog.
  - ☞ The number of records in each relation, the size of the records, whether an index is available, etc.
- The cost to execute a query also depends on the size of the database buffer in main memory.
  - ☞ We often use worst case estimates, assuming only the minimum amount of memory needed for the operation is available.
- For simplicity we use *number of page I/Os* as the cost measure.
  - ☞ **We will ignore the difference in cost between sequential and random I/O as well as CPU cost.**
    (Real query processors take these factors into consideration.)
  - ☞ **We normally do not include the cost of writing the final output to secondary storage.** Why?

SELECTION: EQUALITY SEARCH

Algorithm A1: Linear Search

Linear search is always applicable.

Cost estimate on candidate key $\Longrightarrow$ retrieves a single record.

- $B/2$ since we stop when we find the record.

Cost estimate on non-candidate key $\Longrightarrow$ retrieves multiple records.

- B

Algorithm A2: Binary Search

Applicable if the selection is on the attribute on which the *file is ordered*, *and* the *pages are stored contiguously*.

Cost estimate on candidate key $\Longrightarrow$ retrieves a single record.

- $\lceil \log_2(B) \rceil$

Cost estimate on non-candidate key $\Longrightarrow$ retrieves multiple records.

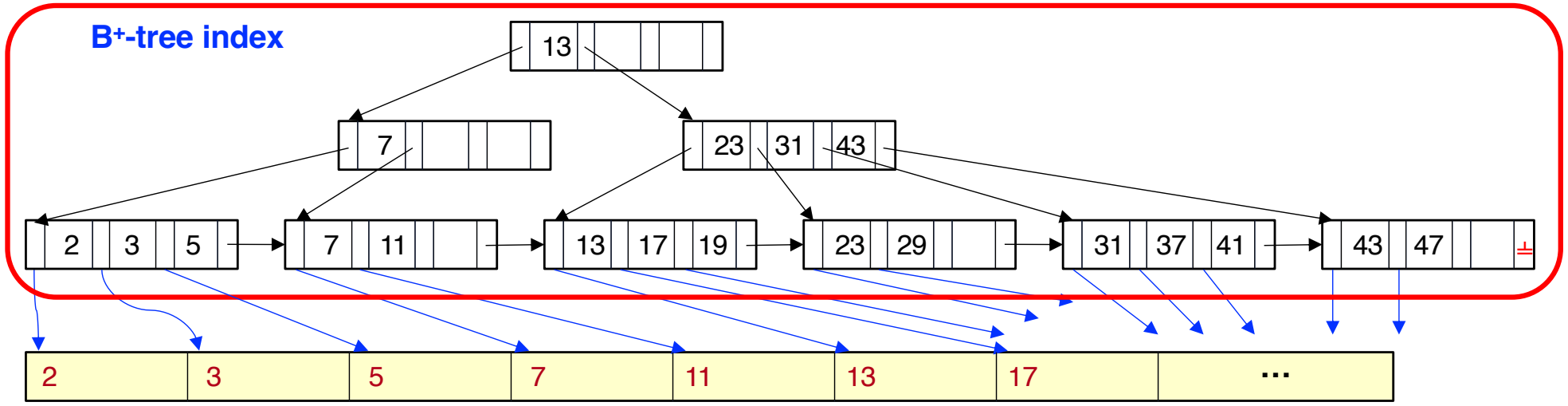
- $\lceil \log_2(B) \rceil$ for locating the first record satisfying the selection condition
  + # pages containing records satisfying the selection condition.

SELECTION: EQUALITY SEARCH (CONT'D)

Algorithm A3: **Using A Clustering Index** ⟹ (data file ordered on search key)

Cost estimate on candidate key ⟹ retrieves a single record.

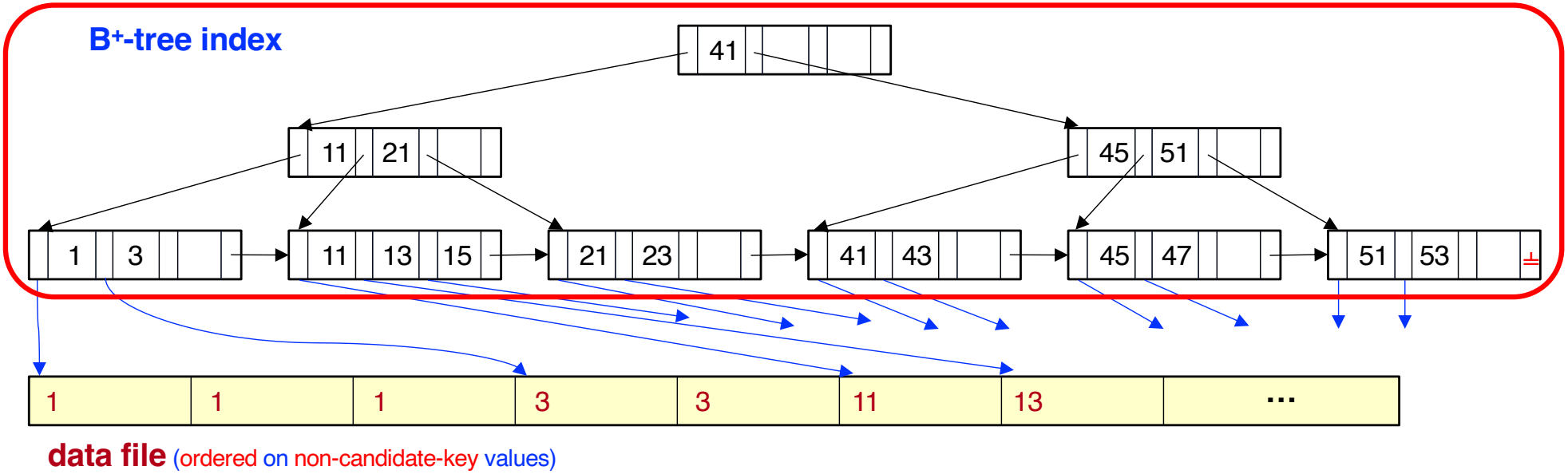
- For a B⁺-tree index: $HT_i + 1$ where HT_i is the height of the index.



SELECTION: EQUALITY SEARCH (CONT'D)

Cost estimate on non-candidate key ⟹ retrieves multiple records.

- For a B⁺-tree index: HT_i + # pages with records that satisfy the selection condition (we usually assume that each record requires a page access).

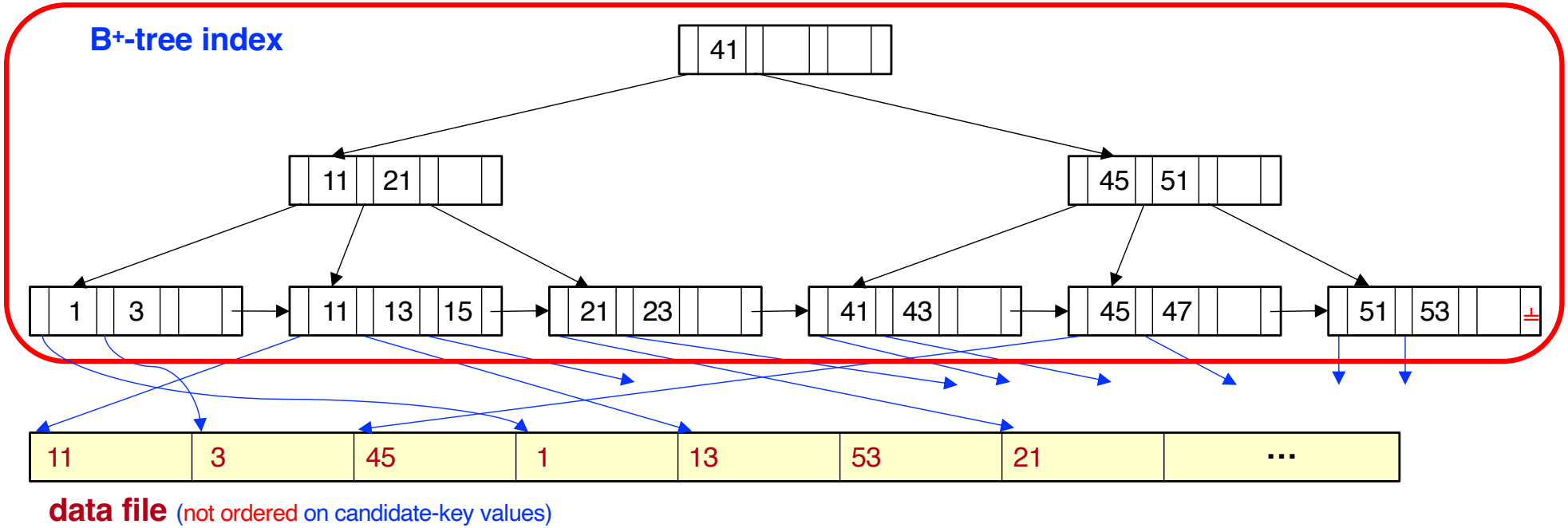


SELECTION: EQUALITY SEARCH (CONT'D)

Algorithm A4: Using A Non-clustering (Secondary) Index

Cost estimate on candidate key ⟹ retrieves a single record.

- For a B⁺-tree index: $HT_i + 1$ where HT_i is the height of the index.
- For a hash index: 1 + 1 *or* 1.2 + 1 if there exist overflow pages.

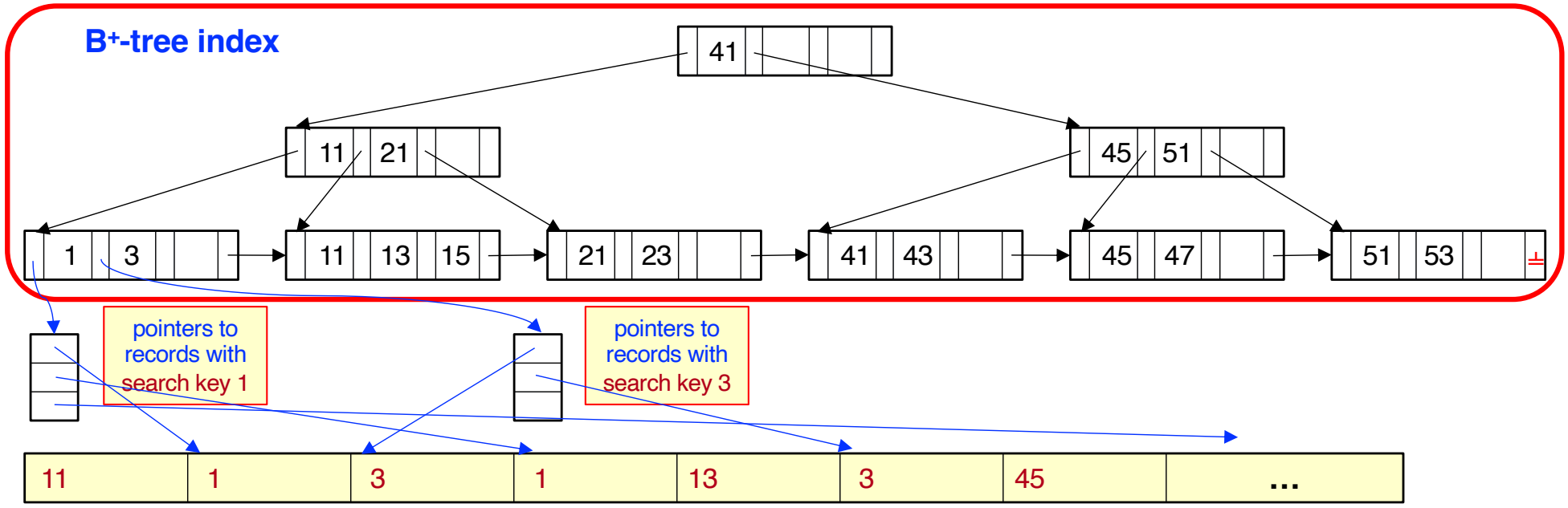


SELECTION: EQUALITY SEARCH (CONT'D)

Cost estimate on non-candidate key ⟹ retrieves multiple records.

- For a B⁺-tree index: HT_i + cost to retrieve indirection pointers + # records retrieved (assumes each record requires a page access).
- For a hash index: 1 + cost to retrieve indirection pointers + # records retrieved.
  or 1.2 + cost to retrieve indirection pointers + # records retrieved (if there exist overflow buckets).

☞ **Can be very expensive!** Why?



SELECTION: RANGE SEARCH

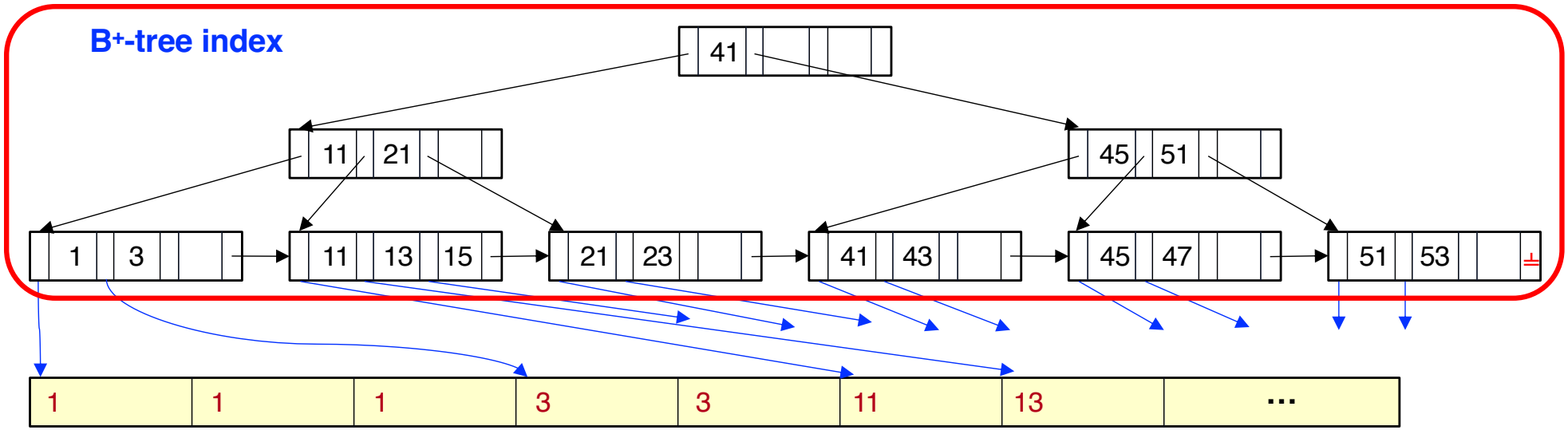
$\sigma_{A \geq v}(r)$ *or* $\sigma_{A \leq v}(r)$

Use linear search, binary search or indexes as follows.

Algorithm A5: **using a clustering B⁺-tree index, file sorted on A**

$\sigma_{A \geq v}(r)$ use the index to find the first record where $A \geq v$ and scan the file sequentially from there.

$\sigma_{A \leq v}(r)$ do not use the index; instead, scan the file sequentially until you find the first record where $A > v$.



data file (ordered on non-candidate-key search-key values)

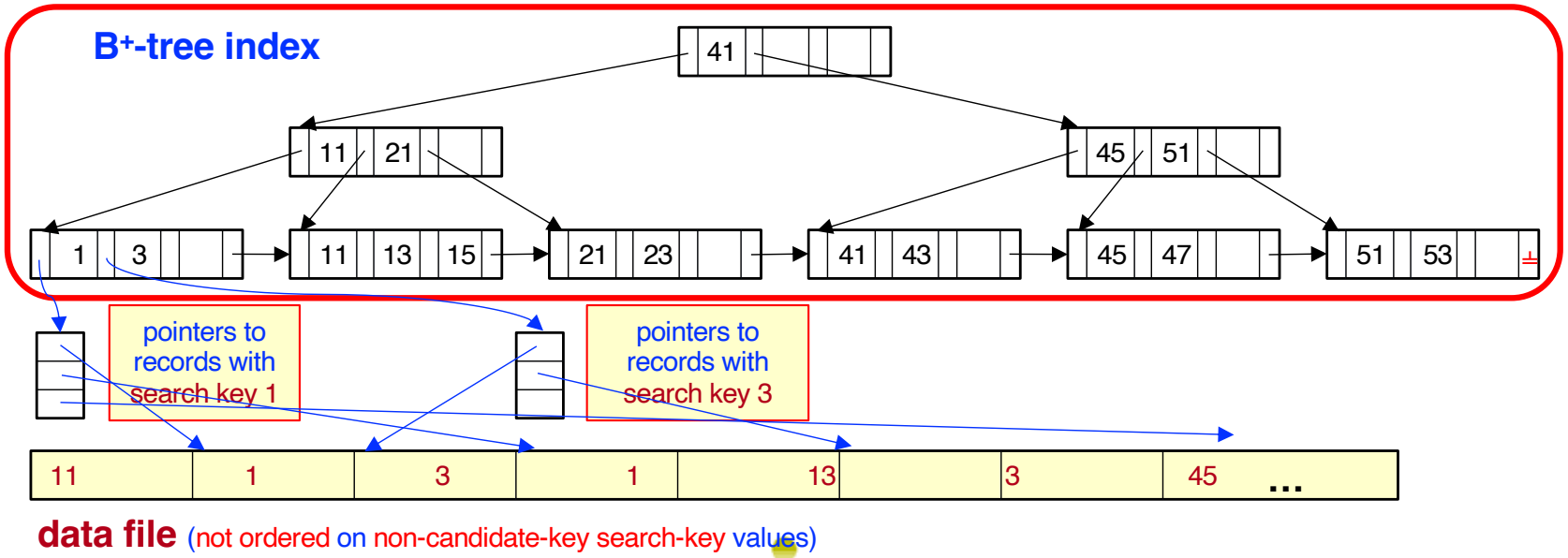
SELECTION: RANGE SEARCH (CONT'D)

$\sigma_{A \geq v}(r)$ *or* $\sigma_{A \leq v}(r)$

Algorithm A6: **using a non-clustering (secondary B+-tree index)**

$\sigma_{A \geq v}(r)$ use the index to find first index entry where $A \geq v$ and scan the leaf pages of the index sequentially from there to find pointers to the records.

$\sigma_{A \leq v}(r)$ use the index to scan the leaf pages of the index sequentially finding pointers to records, until the first entry where $A > v$.



SELECTION: COMPLEX PREDICATES

Conjunction (AND): $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$

Algorithm A7: using one B+-tree index

- Select a θ_i and algorithms A1 through A6 that results in the least cost for $\sigma_{\theta i}(r)$.
- *Check the remaining conditions in the record after retrieving it into buffer (memory).*

Algorithm A8: using a composite B+-tree index

- If available, use a composite (multi-attribute) index (i.e., a combination of θ_i).
- The type of index determines which of the algorithms A3, A4, A5 or A6 will be used.

SELECTION: COMPLEX PREDICATES (CONT'D)

Conjunction (AND): $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$ *or* **Disjunction (OR):** $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n}(r)$

Algorithm A9: conjunction by intersection of record pointers

If *any attributes* of the individual conditions have indexes with record pointers, then use corresponding indexes and intersect record pointers. **Else** use linear search.

- For conditions that do not have appropriate indexes, *check the conditions in the record after retrieving it into the buffer (memory)*.

Algorithm A10: disjunction by union of record pointers

If *all attributes* of the individual conditions have indexes with record pointers, then use corresponding indexes and union record pointers. **Else** use linear search.

- For both cases, use the record pointers to retrieve the records from the file.

  Note: In some cases, only an index scan is required (e.g., count).

Cost estimate for complex predicates

- Using indexes: sum of the costs of the individual index scans
  + the cost of retrieving the records.

SELECTION: NEGATION

Negation (NOT): $\sigma_{\neg\theta}(r)$

- Use linear search.
- If very few records satisfy $\neg\theta$ *and* an index is applicable for θ then
  - Use the index to find records satisfying the negation and retrieve the records from the file.

Example

$$\sigma_{\neg(\text{branchCity}<\text{'Brooklyn'})}(\text{Branch})$$

Suppose that a B^+-tree index on branchCity is available on relation Branch, and that no other index is available.

Use the index to locate the first record whose branchCity field has value "Brooklyn".

From this record, follow the pointer chains in the index until the end, retrieving all the records.

SORTING IN A DBMS

- When does a DBMS need to sort data?
  - To order query results (e.g., increasing by age).
  - To remove duplicate results.
  - For bulk loading a B^+-tree index.
  - Before performing a join operation (e.g., merge-join).
- Often database data is too large to fit into available main memory all at once.

☞ **External sorting is often required.**

EXTERNAL SORTING (CONT'D)

Continuing with the previous example:

Question: We assumed that each file is already sorted. If a file is not sorted, how do we sort it (using only the 3 buffer pages)?

Answer: Each file in the example is only 2 pages. Therefore, we can bring each file into memory and sort it using any main-memory sorting algorithm before doing the merge.

Question: The previous example assumes two separate files. How do we apply this idea to sort a single file?

Answer: Split the file into two parts and merge them as if they were separate files.

Question: Can we do better if we have $M > 3$ main memory pages?

Answer: Yes. Instead of a 2-way merge we can merge up to M-1 files (because we need 1 page for writing the output).

EXTERNAL SORT-MERGE ALGORITHM (CONT'D)

- Let M denote the buffer (memory) size (in pages), i the number of runs (i.e., the number of files to be merged).
- If $i \geq M$, several merge *passes* are required.

  In each merge pass, contiguous groups of M - 1 runs are merged.
- A merge pass reduces the number of runs by a factor of M -1 and creates runs longer by the same factor.

  E.g., If M=11, and there are 90 runs, one merge pass reduces the number of runs to $\lceil 90/10 \rceil = 9$, each 10 times the size of the initial runs.
- Merge passes are performed until all runs are merged into one.

Number of passes (sort & merge)

- $1 + \lceil \log_{M-1}(B/M) \rceil$ passes where B is the file size in pages

I/O cost (sort & merge)

- $2*B + 2*B*\lceil \log_{M-1}(B/M) \rceil = 2*B*(1 + \lceil \log_{M-1}(B/M) \rceil)$ pages

  ($2*B$: sort; $2*B*\lceil \log_{M-1}(B/M) \rceil$: merge)

EXTERNAL SORT-MERGE ALGORITHM (CONT'D)

- Example assuming the number of pages to sort B=12 with 1 record per page and M=3 buffer pages.

- **Number of passes**

  $1 + \lceil \log_{M-1}(B/M) \rceil = 3$

  sort pass; merge passes

- **Number of page I/Os** (read and write):

  Pass 0 (create sorted runs):
  12 x 2 = 24 (12 pages; 1 read, 1 write for each page)

  Pass 1 (merge): 12 x 2 = 24

  Pass 2 (merge): 12 x 2 =24

  Total I/O cost = 72 pages

  ☞ $2*B*(1+\lceil \log_{M-1}(B/M) \rceil) =$
  $2*12*(1+\lceil \log_2(4) \rceil) = 72$